Hardening the Exact-Cell KV Window for `--kv q4lm8v`: Design, Decisions, and a Measurement-First Validation

Matthias Kronenberger

October 2026

Abstract

The exact-cell KV window is an opt-in memory/accuracy point for Strata's 4-bit Lloyd KV cache (`--kv q4lm8v`). A small resident ring of recent cells and a set of sink cells are kept in fp16 while the rest of the cache remains 4-bit. With S=256 recent cells, R=64 sinks and the automatic prompt chunk capped at B=6,144, the production point uses 5.29 bit/cell (1,016 MiB at 128K) against `int8`'s 8.25 bit/cell (1,584 MiB), at equal prefill throughput and decode inside run-to-run spread. This paper describes the design, the correctness hardening (A1–A5), and a measurement-first validation. Quality is scored teacher-forced against an fp16 KV reference on three fixed corpora (2K, 4K, 128K), reporting mean KL, p99 KL, top-1 agreement and perplexity, with paired per-segment statistics. The window removes most of the plain 4-bit base's error on the cwe task: 73% of its gap to `int8` at 4K and 58% of its mean KL at the 128K tail. At 2K its mean KL is not statistically distinguishable from `int8` (0.00702 against 0.00503, paired $t = -1.92$); at 4K `int8` is significantly better on the mean (paired $t = -3.55$) but indistinguishable in the answer region; at 128K `int8` keeps the better p99, the last 1,024 positions and the whole segment. The result is a 36% KV-memory saving with a small, measurable KL cost, not an accuracy win over `int8`. The `int8` default path stays byte-identical in 8/8 runs. AMD HIP is measured; CUDA/SYCL builds and a full cell-to-row parity test remain review gates.

1 Introduction

At long context the KV cache is a large, fixed cost: at 128K tokens the `int8` default reserves 1,584 MiB before any expert weights are loaded. A 4-bit KV cache reduces that, but quantizing every cell uniformly costs accuracy on long documents: earlier 4-bit KV work measured document perplexity 8–12% above the 8-bit default at 1K and 8K context [16, 14]. The exact-cell KV window is an attempt to keep the memory saving while protecting the cells that matter most.

The memory landscape assumed here is not the datacenter one. On Hopper- and Blackwell-class NVIDIA hardware, an FP8 KV cache is the production two-times point: vLLM reports at most 1–2 accuracy points of degradation from FP8 KV-cache and attention quantization, sustained on prompts up to a million tokens [13]. A consumer AMD GPU driven through HIP has no FP8 attention path; on such hardware the block-quantized `int8` default is the deployed two-times saving, and 4-bit is the frontier. That is the constraint the window addresses.

The idea is simple. The most recent S cells and the first R cells (attention sinks) of each attention layer are kept in fp16 in a resident side pool; older cells stay in the 4-bit Lloyd format. The window thus spends exact bytes where the model is most sensitive and leaves the long tail compressed. It is opt-in (`--kv q4lm8v` plus `STRATA_KV_WINDOW=1`) and never changes the default `int8` path.

The implementation was functional in principle, but several integration points could fail silently: the host-side prompt chunking, the interaction with KV streaming, the stability of the window grid across model states, and the meaning of uninitialized ring entries. We treated these as correctness issues, not performance issues, and fixed them behind the same opt-in flag.

This paper makes six contributions. It positions the window against prior and current KV-cache compression work (Section 2); it describes the window design and its production point (Section 3); it records the alternatives weighed against this design and why each was left out (Section 4); it documents the hardening decisions and the evidence that the default path is untouched (Section 5); it sets out the measurement methodology — mean KL, p99 KL, top-1 agreement and perplexity, with paired statistics (Section 6) — and reports results at 2K, 4K and 128K context (Section 7), with a discussion of where the point fits (Section 8). The emphasis is on the methodology and on what the measurements do and do not show; file-level implementation detail is deliberately kept out.

2 Related Work

KV-cache compression splits into *eviction* and *quantization*. Eviction keeps a subset of cells and drops the rest: H2O [7] accumulates attention as heavy hitters, SnapKV [5] selects per-head clusters from an observation window, and PyramidKV [6] allocates larger budgets to lower layers. Quantization keeps every cell at a lower bit width: KIVI [2] shows tuning-free 2-bit per-channel K and V with a fixed fp16 residual of recent tokens; KVQuant [3] adds per-channel keys, pre-RoPE key quantization and non-uniform datatypes to reach sub-0.1 PPL degradation at 3 bits; GEAR [4] corrects residual error with a low-rank plus sparse-outlier decomposition. Rotations make low-bit quantization robust to channel outliers: QuaRot [8] and RotateKV [9] apply orthogonal or Hadamard rotations before quantization. Strata's 4-bit base quantizes per cell after exactly such an orthonormal Walsh–Hadamard rotation, so it belongs to that family; the K-is-more-sensitive-than-V asymmetry behind its 4-bit K / 8-bit V split is the

System	Point	Documented outcome
KIVI [2]	2b K, 2b V	per-channel, fp16 recent residual; tuning-free
KVQuant [3]	3b per-channel + pre-RoPE	<0.1 PPL degradation (Wikitext-2/C4)
GEAR [4]	4b + correction	low-rank + sparse
RotateKV [9]	2b	rotation-based
KVTuner [10]	3.25–4.0b	layer-wise pairs, nearly lossless
SnapKV [5]	eviction	per-head clusters
PyramidKV [6]	eviction	12% of cache matches full cache (LongBench)
`llama.cpp` [12]	q8_0 / q4_0	block-quantized, per-cache type
`vLLM` [13]	FP8	≤1–2 acc. points up to 1M-token prompts

Table 1: Documented operating points from the literature and production runtimes. All entries are the cited works' own published results on their own hardware and models; they are not comparable across rows.

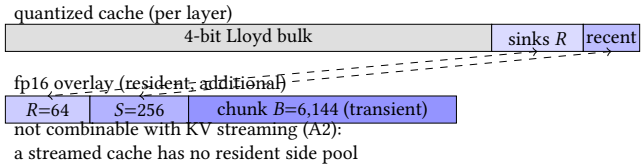


Figure 1: Cache anatomy per layer, widths not to scale (R and S are tiny against 131,072 cells). Every cell stays quantized in the 4-bit base; the copied fp16 rows form an *additional* resident side pool covering the R sink cells, the S most recent cells and the current prefill chunk.

KV mode	Bits/cell	KV at 128K
`int8` (default)	8.25	1,584 MiB
plain `q4lm8v`	4.50	864 MiB
`q4lm8v` + window 256/64	5.29	1,016 MiB

Table 2: KV footprint at 128K (131,072 cells, 12 attention layers). `int8` stores 8-bit content plus block scales; its 8.25 is the effective rate. The window figure includes the resident ring at B=6,144; the plain figure is from the byte model.

same observation KVTuner [10] formalizes with layer-wise precision pairs. For token-precision co-tuning across layers, [11] studies the trade-off between kept tokens and bits per token. Table 1 collects the documented operating points in one view.

In production, `llama.cpp` ships block-quantized KV (q8_0, q4_0) selected per cache [12], and `vLLM` ships an FP8 KV-cache path whose measured degradation stays within 1–2 accuracy points on reasoning and long-context benchmarks on Hopper- and Blackwell-class hardware [13]. Strata targets a corner where neither path exists—one consumer GPU through HIP, no FP8 attention kernels—so its block-quantized `int8` default already provides the two-times point, and this paper moves toward 4-bit.

The window itself is a quantization-side design: it keeps the whole context addressable (no eviction, unlike [7, 5, 6]), and it spends fp16 bytes on a small recent-plus-sink overlay. The recent-token residual is KIVI's [2] and the sink retention is StreamingLLM's [1]; what this paper adds is the overlay on top of a block-quantized Lloyd base on this engine, the hardening around it, and the paired-statistics discipline used to read it. The fair competitor we do not measure here is a per-channel-K arm in the KIVI/KVQuant style on the same engine; it is listed as follow-up work.

Code and documentation for the compared systems are listed with the references [1, 2, 3, 10, 12, 13]. Strata's own measurement provenance lives in the repository and is described in Section 6.6.

3 Design of the Exact-Cell Window

Throughout, a *cell* is one token's K or V entry in one attention layer; the reference model has 12 attention layers. A *row* is that cell's storage within the layer's cache, and the mean numbers of this section (bits/cell, MiB) are per layer and per context token unless stated otherwise. `int8` names the default's block format: 8-bit content plus block scales, 8.25 effective bit/cell.

3.1 Operating point

The production point is S=256, R=64 and the automatic prompt chunk capped at B=6,144. Table 2 compares it with the default. The window costs 0.79 bit/cell more than the plain 4-bit base (4.50 bit, 864 MiB) and frees 36% of the KV memory against `int8`.

3.2 Window geometry and cost

The 4-bit base stores K at 16 levels and V at 8 levels in 18-byte blocks, 576 B per cell and layer. The window keeps a side copy of the sink and recent cells in fp16. The side copy is *additional* to the quantized row, not a replacement: the exact cell is 16 bits, not 16 − base bits. The resident ring holds R sink rows, S recent rows and B rows for the current prompt chunk, so the marginal cost of a windowed run is

$$\text{bits/cell} \approx 4.50 + \frac{16\,(R+S+B)}{L}, \tag{1}$$

where L is the context length in cells. All three terms are resident memory: the ring is sized for the prompt chunk, so the B term dominates the per-cell average at this L (0.75 of the 0.79 added bit/cell at 128K). At 128K with R=64, S=256 and B=6,144 this gives 5.29 bit/cell and 1,016 MiB.

The chunk B follows `--prefill`. A larger automatic chunk would need a larger ring, and the ring's scan limit is 6,144 cells, so a windowed run caps the automatic chunk there and says so. A ring the user sizes explicitly is left alone; a chunk that does not fit is refused loudly. Below roughly 32K cells the ring costs more than the 4-bit base saves, which is why setup offers the window only from 32K upwards.

The exact cells must stay resident. A streamed KV cache has no always-resident side pool, so the window and KV

streaming do not combine (A2). The window is also orthogonal to the level table: it can be enabled on other 4-bit modes, but the production point is `q4lm8v` with 256/64.

4 Design Alternatives Considered

None of the alternatives in this section was measured on this engine. The section records why each was left out; the reasoning is static analysis, not measurement, and every item is a candidate follow-up arm.

Per-channel key quantization. KIVI and KVQuant quantize K along the channel axis because the key error concentrates in outlier channels [2, 3]. Strata's base quantizes per cell after an orthonormal Hadamard rotation, which spreads that outlier energy across channels; whether the rotation makes the cheaper per-cell axis competitive with the per-channel axis on this model is exactly the unmeasured question. It would change the block writer and reader, not the window.

Pre-RoPE key quantization. KVQuant quantizes K before the rotary positional embedding, because the rotation multiplies the quantization error [3]. Strata's Hadamard rotation is also pre-quantization; whether the engine's K path can quantize before RoPE instead of after is a kernel-layout question that has not been examined.

Non-uniform levels. KVQuant derives sensitivity-weighted datatypes [3]; our K uses 16 uniform Lloyd levels and V uses 8. Non-uniform levels are orthogonal to the window and would change the level tables only.

Low-rank plus sparse correction. GEAR keeps a correction term for the residual error [4]. The window's fp16 overlay is essentially a dense-but-local correction: full 16-bit precision on 4.9% of cells. A distributed low-rank correction would cover the whole cache at a smaller byte rate, at an added latency cost; both would need new kernel shapes.

Layer-wise precision pairs. KVTuner reaches a nearly lossless 3.25-bit point with different precision per layer [10]. Strata's KV format is global per cache; per-layer level tables are representable in the block layout but add configuration space that setup would have to explain to users.

Eviction on top of the 4-bit base. SnapKV/H2O-style eviction is a larger memory win with a different cost: cells leave the cache [5, 7]. The window was designed so the whole context stays addressable — the A-series treats a silently unavailable exact cell as a fault — so eviction is out of scope for this point; a combined mode would have to be a separate opt-in with its own hardening.

A streaming-compatible sink pool. A2 refuses the window under streaming. A tiny sink-only side pool (R cells, no recent ring) could in principle combine with a streamed cache, because sinks never scroll far from position 0. We did not build it: the sink budget interacts with the streaming host copy, and the current contract prefers loud refusal over a new silent-failure surface. Untested.

5 Hardening: Preventing Silent Failures

The guiding principle is that a silent misread is worse than a loud refusal. When the window cannot guarantee an exact read, it should stop, refuse the configuration, or fail an invariant—not fall back quietly to a quantized value that the caller believes is exact. Table 3 summarizes decisions A1–A5, including A4 as the visible open item.

Two decisions deserve emphasis. A2 is enforced twice—once in setup, where most users will meet it, and once in the engine, where a hand-edited configuration can still arrive. A3 is deliberately a freeze-and-assert rather than a full state-layout refactor: it closes the silent-drift hole without moving the default path, and it leaves the larger refactor to the parity test. A4 is the one item in the A-series that is still open.

6 Evaluation Methodology

6.1 Corpora

Three fixed teacher-forced corpora are used. The 2K set is the RULER *complex word extraction* task in eight contiguous segments, 16,351 scored positions. The 4K set uses the same task with eight segments of 4,096 tokens, 32,746 positions. The 128K set is one 131,063-token segment, 131,062 scored positions, scored on the last 4,096 positions and on the whole segment. The three sets are different texts; numbers are not comparable across lengths, and the 4K set is not a length-scaling experiment for the 128K set.

6.2 Reference and scoring

The reference is fp16 K/V with fixed Swift 1.5 IQ3_XXS weights. The model is given the reference text and the log-probabilities are read; there is no free generation. Each arm is a separate engine run, and the harness records per-position log-probabilities and compares them offline, so KL, top-1 and PPL come from the same pass. The KV cache of the prefill is quantized along with the decode cache, so the absolute gaps are internal Strata numbers and are not directly comparable with field numbers measured decode-only. The 2K and 4K runs use one expert slot; the 128K quality runs use one expert slot and `--prefill 512`. The runtime runs use the automatic expert cache and a mixed prompt set.

6.3 Metrics

Four metrics are reported. *Mean KL* is the Kullback–Leibler divergence $D_{\mathrm{KL}}(p_{\mathrm{ref}} \,\|\, p_{\mathrm{arm}})$, from the fp16 reference distribution to the arm's distribution, in nats per position; zero means identical, and lower is better. KL is not symmetric, so the direction is fixed: this direction directly penalizes an

ID	Observed problem	Decision	Evidence
A1	The automatic prompt chunk could exceed the ring's scan limit, so an append in the middle of a long prompt aborted the run.	Cap the automatic chunk at the ring's batch maximum (6,144) when the window is active and the user did not set a size; cover the generate and serve paths.	An 8,192-token prompt completes with a 6,144 chunk and no abort. An explicitly undersized ring is still refused as a contract violation.
A2	Setup could write a streamed KV cache beside the window. A streamed cache has no always-resident side pool, so a promised exact read could silently become quantized.	Setup never writes the pair and explains why; the engine refuses the pair before loading anything.	The pair is refused in about one second; window alone and `int8` with streaming both run. Setup tests pass 59/59.
A3	The (S, R, B) grid could drift between the main state and a later-carved MTP/twin state. Different grids on the same contract are incorrect.	Freeze the grid after the prompt chunk and assert stability on later batch changes.	The grid stays stable; the default logits SHA is unchanged; targeted probes pass.
A4	The window's write/read paths have no full parity test across all shapes, so a shape-specific misread would be silent.	**Open item.** Dump every window row at write time and re-read it through the window reader on every shape: ring wrap, stale tags, sinks, batch boundaries, empty tags.	Not yet run; the test is specified in the repository and is a review gate.
A5	Zero-initialized ring rows were tagged as cell 0 and could be read as a valid exact cell.	Initialize empty tags to a sentinel; a stale or mismatched tag falls back to the quantized row.	Empty-state semantics are correct and the default path is unchanged.

Table 3: Hardening decisions A1–A5. The A4 row is the open follow-up item (a full cell-to-row parity test across all shapes), a review gate rather than a shipped fix.

arm that puts little probability mass where the reference has mass (bulk distribution drift), which is the kind of error a downstream sampler inherits. The reverse direction weights the arm's own modes instead; the ranking could differ under it, and we have not measured that. *p99 KL* is the 99th percentile of the per-position KL: it exposes a heavy error tail that a good mean can hide. *Top-1* is the fraction of positions where the arm's argmax equals the fp16 argmax; it is a coarse agreement measure, not target accuracy. *Perplexity* is exp of the mean negative log-likelihood of the true tokens. Perplexity is reported for context only: quantized-KV error tends to damp the distribution, so a quantized arm can score *below* the fp16 reference while its $D_{\mathrm{KL}}(p_{\mathrm{ref}} \,\|\, p_{\mathrm{arm}})$ is positive. KL, not perplexity, is the decision metric. The p99 entries are point estimates; at these position counts their sampling error is much smaller than the spread between arms, but it is not quantified here — a bootstrap over positions is the cheapest missing piece and needs no new runs.

6.4 Paired statistics

Both arms see the same positions. We compute the KL difference per segment, then average over the eight segments. The text cancels, so the spread is much smaller than in a comparison of two independent means. We report the paired t statistic and the number of segments whose difference has the expected sign. With eight segments (seven degrees of freedom), $|t| \gtrsim 2.4$ is roughly $p < 0.05$ two-sided and $|t| \gtrsim 3.5$ roughly $p < 0.01$. A large t with 4/8 segments is not a result; the sign count is read alongside it. The 128K set is one segment, so its eight 4K blocks are adjacent and not independent; its t is a rough reading, not a segment-pair test.

6.5 Runtime, memory and correctness gates

Runtime is a same-day A/B with a fresh server per arm and two alternating rounds; the engine's own prefill and decode counters are read, and table entries are medians of the two rounds. The reported runs use a mixed prompt set — short and long prompts over the same serving session — whose per-prompt results are documented in the measurement report [17]; the hit rate column is the expert-cache hit rate over that set, i.e. the fraction of expert lookups served from the resident cache. Reported memory is KV bytes at 128K and resident expert slots. The correctness gate is byte-identity: the `int8` default's dump-logits SHA must be identical across runs, and every window change must leave it unchanged. Targeted probes reproduce the former crash and the refusal cases; kernel parity tests compare the window writer and reader bitwise; needle tests at 32K and 128K check that retrieval does not regress.

6.6 Provenance and reproducibility

Every number in this paper is traceable to the repository's measurement history: the window work is documented in `docs/KV_QUANT_WORKPACKAGE.md` [17], the authoritative byte model and quality section in `docs/DETAILS.md` [14], and the plain-4-bit document-perplexity measurement in `bench/results/2026-09-27-kv-q4` [16]. The cwe corpora are RULER complex-word-extraction texts generated by the repository's quant-quality harness (fixed segment structure and scored positions per section above; the harness and its KL-yardstick conventions are documented in `docs/QUANTS.md` [15]); the harness records per-position log-probabilities that the KL, top-1 and PPL statistics are computed from. No new runs were produced for this paper;

where a figure is byte-model rather than runtime-observed, the table captions say so. The engine, terminal and harness are in the Strata repository; the measurement reports are plain-text files next to the code.

7 Results

7.1 Memory and speed

Table 4 gives the 128K runtime point. The production arm (256/64, B=6,144) has the same prefill throughput as `int8`, decode 1.0 token/s lower with changing signs across rounds (inside the run-to-run spread), and a capacity gain the numbers state directly: it frees 568 MiB of VRAM at 128K, which the expert cache converts into 416 more resident expert slots and a slightly higher expert-cache hit rate. Serving throughput at concurrency was not measured, so these are capacity figures, not speed wins. The B=512 arm is cheaper (4.60 bit, 884 MiB) but prompts at 105 token/s, about five times slower, so it is an offline/batch point. The 512/128 arm is not chosen: its quality is the same as 256/64, it costs 8–10 MiB more, decodes about 0.85 token/s slower, and its apparent prefill advantage did not reproduce on the needle prompts.

7.2 Quality at 2K and 4K

Tables 5 and 6 report the short-context quality; all quality statements in this section are for the cwe task and this model, and are not length-scaling claims across the different corpora. At 2K the window's mean KL is 0.00702 against `int8`'s 0.00503, but the paired difference is not significant ($t = -1.92$, 2/8 segments positive). The window is also close in the tail: p99 0.0835 against 0.0733. It removes most of the plain 4-bit arm's error (0.01791) and lands near `int8`.

At 4K `int8` is significantly better on the mean (0.01688 against 0.02090; paired $t = -3.55$), and the window closes 73% of the plain-arm-to-`int8` gap (0.01093/0.01494). In the small answer region the two are not distinguishable ($t = -0.51$; descriptive, small region). The 512/128 form is statistically identical to 256/64 ($t = 0.02$), so the smaller form is kept.

7.3 Quality at 128K

Table 7 reports the tail of the 128K run, and Table 8 the other regions. On the last 4,096 positions the window's mean KL is 0.00663 against `int8`'s 0.00959, a 31% lower mean, but the paired eight-block t is −0.74: the lead is not significant and comes from two blocks in which `int8` has its own spikes. The robust reading favors `int8`: it has the better p99 (0.01655 against 0.02977), the better last 1,024 positions (0.00265 against 0.00444), and the better whole segment (0.10715 against 0.13312); the region table also shows a lower answer-region KL for `int8`, but that region has 27 positions, so the number is descriptive only and we do not use it as an argument. The window still beats the plain 4-bit base clearly: mean KL 0.00663 against 0.01587 (58% lower), whole segment 0.13312 against 0.15758, and p99 0.0298 against 0.0308. The 128K result is therefore the same story as 4K: the window recovers much of the base's loss and approaches `int8`, but does not overtake it.

7.4 Reading the results

Three conclusions follow, all within this task family and this model. First, the window does what it was designed to do: it removes most of the plain 4-bit base's error at every length tested, and at 2K it is statistically indistinguishable from `int8` on cwe-2k. Second, it does not replace `int8` as the quality default. At 4K `int8` is significantly better on the mean; at 128K `int8` keeps the p99, the last 1,024 positions and the whole segment. The window saves 36% of the KV memory at the price of a small, measurable quality gap that the mean KL alone can hide. Third, perplexity is not a useful discriminator here: the 128K PPL values (2.4052, 2.4079, 2.4029) all sit below the fp16 reference (2.4149) while every KL is positive. Reporting PPL without KL would reverse the ranking.

7.5 Correctness gates

The default path is unchanged: the `int8` dump-logits SHA is byte-identical in 8/8 runs (`A014F504. . . BA052`). The former crash config now completes with a 6,144 chunk; an explicitly undersized ring is still refused; the window-plus-streaming pair exits with code 2 in about one second. Setup tests pass 59/59, kernel parity tests pass 12/12, and needle retrieval is 18/18 at 32K and 128K (6/6 per arm; the test is saturated and shows no regression but no separation). A full cell-to-row parity test remains open (A4).

8 Discussion: Where the Point Fits

8.1 A decision rule

The measured numbers compose into a short rule for operators. If context stays below roughly 32K cells, the window is off the table: the ring costs more than the 4-bit base saves (Section 3.2). At long context, quality-first workloads keep `int8`: it wins or ties every region statistic that carries weight (Tables 5–7). Memory-bound workloads at 128K take the window: 568 MiB freed, 416 more resident expert slots, prefill parity, decode inside spread (Table 4), and a measured KL penalty that is small on this task family and bounded by the 4K and 128K readings. Setup already follows this shape: `int8` is the default, the window is opt-in from 32K upwards.

8.2 How firm is the grid

The neighborhood of the production grid is measured more thinly than the endpoints. On cwe-2k the window first carries the load near S=512, and the §7.16 knee arms (R=64 with S=256, plus the 512/128 sink pair) are each statistically indistinguishable from `int8` on both bases [17]. At 4K and at 128K the 512/128 pair scores the same quality as 256/64 (Tables 5 and 7) for 8–10 MiB more bytes and a slower decode, which is why 256/64 ships. What was not measured at 128K is anything smaller than 256/64: the data support 256/64 as the cheapest measured arm with indistinguishable quality, not as a local optimum. A finer sweep of S and R at 128K — and the knee move observed at 2K, where the load-bearing S sits higher — is the natural next measurement.

8.3 Cheaper overlays exist on paper

The window is orthogonal to the level table, and the repository documents a ≤3-bit K overlay on the same mechanism

Arm	Bit	MiB	Slots	Prefill	Decode	Hit
int8	8.25	1,584	3,033	526.1	**35.2**	0.74
256/64, B=6,144	5.29	1,016	3,449	528.0	34.2	0.76
256/64, B=512	**4.60**	**884**	3,496	105.0	34.5	0.76
512/128, B=6,144	5.33	1,023	3,412	**634.0**	33.35	0.77

Table 4: 128K runtime (Swift 1.5 IQ3_XXS, RX 9060 XT 16 GB, Windows HIP). Prefill and decode in tokens/s; medians of two same-day alternating rounds over a mixed prompt set. The 512/128 prefill advantage did not reproduce on the needle prompts and is shown for completeness.

Arm	Top-1	Mean KL	p99 KL	PPL
fp16 (reference)	—	0	—	2.999
int8	99.35%	**0.00503**	**0.0733**	3.004
plain q4lm8v	98.92%	0.01791	0.2793	3.024
+ window 256/64	99.33%	0.00702	0.0835	3.003
+ window 512/128	**99.39%**	0.00623	0.0743	**2.995**

Table 5: Quality on cwe-2k (16,351 positions, fp16 K/V reference). Top-1 is agreement with the fp16 argmax; KL in nats/position.

Arm	Top-1	Mean KL	p99 KL	PPL
int8	**99.05%**	**0.01688**	**0.1888**	2.860
plain q4lm8v	98.40%	0.03182	0.5785	2.888
+ window 256/64	98.84%	0.02090	0.2811	2.858
+ window 512/128	98.95%	0.02088	0.2822	2.858

Table 6: Quality on cwe-4k (32,746 positions). cwe-4k is a different text from cwe-2k; the two tables are not comparable across sets.

with a measured byte/quality crossover between roughly 4.8 and 5.3 bit/cell on the small-context grid [17]. That is a documented result, not a validated production point: it ran on the Swift 2K/4K harnesses, under the R=8 grid of those arms, and nothing at this crossover was carried through the 128K runtime or quality gates that Section 7 reports. It is recorded here because it bounds what the mechanism could offer if the memory target tightens further: a leaner base plus the window may match this point's quality at fewer bytes, and proving or refuting that at 128K is a concrete follow-up arm.

8.4 Two unmeasured workload questions

Two questions bound how the point is used today. First, multi-turn growth: the window's home workload is a chat that grows by small appends to a long context, every turn after the first presumably reading large parts of the cache; the quality runs here score single segments, and the runtime A/B's mixed prompt set is the only evidence for repeated prompting. Second, mid-context degradation: the 128K corpus is scored at the tail, and the 4K corpus shows that per-cell quantization hurts where nothing protects the cell; a mid-segment scoring region (for example, around position 64K) would verify that the tail hotspot is not hiding an interior one. Both are unmeasured; neither argues for removing the window, but both belong on the list before larger deployment claims.

9 Threats to Validity

The measurements come from one GPU (RX 9060 XT, gfx1200), one operating system (Windows 11), one ROCm/HIP stack (7.1) and one weight set (Swift 1.5 IQ3_XXS). CUDA and SYCL were not built; the changes are backend-agnostic, but that is an argument, not a measurement. The short-context results have eight segments; the 128K result is one text with adjacent blocks, so its t values are descriptive. The 4K exact-cell fraction is 20.3% at B=512, while at 128K the production ring covers 4.9% of cells; the 4K KL is an upper bound on the long-context benefit, not a prediction, and the 128K KL is measured separately. The prefill KV is also quantized, so the absolute KL gaps are not comparable with decode-only field numbers. Finally, the window is a memory trade-off, not a quality improvement over int8; the paper should be read that way. Two coverage limits stand out. The quality evidence comes from one task family (RULER complex word extraction) on one model; the full RULER set or LongBench subsets were not run, so the quality sentences here carry no claim outside this family. And no community-standard arm — for example a KIVI-style per-channel-K quantization with an fp16 residual [2, 3] — was measured on this engine, so the 5.29-bit point has no external anchor on the published trade-off curve. One intermittent engine hang was seen once on the *plain* 4-bit arm (no window) during a 128K prefill; it did not reproduce in 6/6 reruns, and the window arms completed three 128K runs without it. It remains an open engine bug report, not evidence against the window.

10 Follow-up Work

Two gates remain. A full cell-to-row parity test should dump every window row at write time and re-read it through the window reader on every shape: ring wrap after scrolling past R rows, stale-tag fallback, sink rows, batch-boundary shapes and the empty-tag case. The second gate is build coverage: CUDA (nvcc) and Intel SYCL (dpcpp) should compile and pass their parity suites. q4lm8v is not offered on SYCL, so the window paths are dead there, but they should still compile.

Beyond the gates, the next measurement round has six limbs. Three need GPU time: a per-channel-K arm in the KIVI/KVQuant style on the same engine (the fair community competitor this paper does not include, Section 4); a second RULER task family at 4K, since the quality evidence above covers only cwe; and a finer S/R sweep at 128K including

Arm	Bit @128K	KV MiB	Mean KL	p99 KL	Top-1	PPL
int8	8.25	1,584	0.00959	**0.01655**	**99.73%**	2.4052
256/64	**5.29**	**1,016**	**0.00663**	0.02977	**99.73%**	2.4079
plain q4lm8v	4.50	864	0.01587	0.03079	99.51%	2.4029

Table 7: Quality on cwe-128k, last 4,096 positions (context 126,967–131,062). The fp16 reference has KL 0 and PPL 2.4149; target-token top-1 is 79.74%. This p99 covers the tail region only and is not comparable with the whole-corpus p99 of Tables 5 and 6; over the whole 128K segment the p99 is 3.66–5.55 at all three arms (Table 8), driven by isolated hard positions. The 864 MiB of the plain arm is from the byte model, not a runtime reading.

Region / metric	int8	256/64	plain
Last 1,024, mean KL	**0.00265**	0.00444	0.01096
Last 1,024, p99 KL	**0.04056**	0.06283	0.13641
Whole segment, mean KL	**0.10715**	0.13312	0.15758
Whole segment, p99 KL	**3.65623**	4.75603	5.55046
Answer region (27), mean KL	**0.00526**	0.00861	0.01155
Answer region, top-1	**96.30%**	92.59%	**96.30%**

Table 8: cwe-128k by region. The KL rows are fp16-reference KL; *the top-1 row is target-token accuracy, not the fp16-argmax agreement of Table 7*, and is computed over 27 positions: $25/27 = 92.59\%$, $26/27 = 96.30\%$ — a difference of one position, not a result. The whole-segment p99 values are dominated by isolated hard positions (the measurement report lists single-position KLs of 17.3, 17.1 and 15.8); int8 reaches 3.66 as well, so they do not separate the arms.

pairs smaller than 256/64, because the current data support 256/64 as the cheapest measured arm, not as a local optimum (Section 8.2). Two more score existing corpora differently: a mid-segment scoring region around position 64K on the 128K text, and a multi-turn append A/B where short prompts grow a long context turn by turn (Section 8.3). The last limb is offline arithmetic: a bootstrap over positions for the p99 entries, which needs no engine runs (Section 6.3). Optionally, the $\leq$3-bit overlay crossover at 128K (Section 8.4) rounds out the byte side. Both gates are review items, not shipped claims.

11 Conclusion

Hardening the exact-cell window was primarily an exercise in preventing silent failures at integration boundaries. Setup-side prevention and engine-side enforcement close the window-plus-streaming hole; a frozen grid closes state drift; sentinel tags close the empty-cell misread; a capped automatic chunk closes the mid-prompt abort. The default int8 path stays byte-identical. The measurement-first validation shows the intended trade-off clearly: on the cwe corpora, the window removes most of the 4-bit base's KL error (73% of its gap to int8 at 4K, 58% of its mean KL at the 128K tail), approaches int8 at 2K, and saves 36% of the KV memory at 128K. It does not beat int8 in the tail, and the p99 and whole-segment numbers say so. That is the correct result for an opt-in memory point: keep int8 as the quality default, offer the window where memory is the constraint, and measure both.

Two habits generalize beyond this feature. First, correctness is enforced where the failure would be silent: setup-side for users, engine-side for hand-edited configs, and a byte-identity gate for everything the default path touches. Second, an opt-in point earns its existence only against measured alternatives — here, against int8 in regions that the mean KL alone cannot see. Both habits cost little and transfer to the next integration-boundary feature.

References

[1] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis. *Efficient Streaming Language Models with Attention Sinks.* arXiv:2309.17453, 2023 (ICLR 2024). Code: https://github.com/mit-han-lab/streaming-llm.

[2] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, X. Hu. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.* ICML 2024, arXiv:2402.02750. Code: https://github.com/jy-yuan/KIVI.

[3] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami. *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization.* NeurIPS 2024, arXiv:2401.18079. Code: https://github.com/SqueezeAILab/KVQuant.

[4] H. Kang et al. *GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative LLM Serving.* arXiv:2403.05527, 2024.

[5] Z. Zhang et al. *SnapKV: LLM Knows What You Are Looking for Before Generation.* NeurIPS 2024, arXiv:2404.14469.

[6] Z. Cai et al. *PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling.* COLM 2025, arXiv:2406.02069.

[7] Z. Zhang et al. *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.* NeurIPS 2023, arXiv:2306.14048.

[8] *QuaRot: Outlier-Free 4-Bit Inference in LLMs via Rotations.* arXiv:2404.00456, 2024.

[9] *RotateKV: Accurate and Robust 2-Bit KV Cache Quantization for Large Language Models.* IJCAI 2025 (proceedings 2025/0690).

[10] X. Li, Z. Xing, Y. Li, L. Qu, H. Zhen, Y. Yao, W. Liu, S. J. Pan, M. Yuan. *KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference.* ICML 2025, arXiv:2502.04420. Code: https://github.com/cmd2001/KVTuner.

[11] *More Tokens, Lower Precision: Towards the Optimal Token-Precision Trade-off in KV Cache Compression.* arXiv:2412.12706, 2024.

[12] ggml-org, `llama.cpp` repository: KV-cache quantization types (`-cache-type-k/v`: fp16, q8_0, q4_0). https://github.com/ggml-org/llama.cpp.

[13] vLLM team, *The State of FP8 KV-Cache and Attention Quantization in vLLM*, vLLM blog, 2026-04-22. https://vllm-project.github.io/2026/04/22/fp8-kvcache.html; quantized-KV feature documentation: https://docs.vllm.ai/en/latest/features/quantization/quantized_kvcache.

[14] Strata, `docs/DETAILS.md`: KV-cache quantization section (byte model, KL measurements, window 256/64 at 128K). In-repo.

[15] Strata, `docs/QUANTS.md`: quant-quality yardsticks (mean KL, KL p99, top-1, perplexity). In-repo.

[16] Strata, `bench/results/2026-09-27-kv-q4`: Q4_0 KV with Hadamard rotation; document perplexity 8–12% above the 8-bit default at 1K/8K. In-repo.

[17] Strata, `docs/KV_QUANT_WORKPACKAGE.md`: KV-quantization workpackage with the window's runtime, quality and parity history (§7.12–§7.21). In-repo, branch `KV_QUANT_research`.